

Data Structures: Assignment-10

Name: Suraj Shah

Admission Number: I24AI012

1. You are given a text file, named "students.txt" that contains students' records. Each Line contains information of a single student in the form of <Student Name, Roll No, Department>.

A. Read the records from the file into an array of structures.

B. Three Options will turn up: (1) Bubble Sort, (2) Binary Search, and (3) Quit. In the following we describe what your C/C++ program will do on Selecting the options.

(1) Bubble Sort: Sorts the records based on Student Name. If more than One students has the same name, then sort them on their roll no.

(2) Binary Search: Given a student name, the function will return all the Student records <Student Name, Roll No, Department> having the Student name.

(3) Quit: Exit the program.

Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_STUDENTS 100
#define NAME_SIZE 50
#define DEPT_SIZE 30

typedef struct {
    char name[NAME_SIZE];
    int rollNo;
    char department[DEPT_SIZE];
} Student;

int readFromFile(Student students[], const char* filename) {
    FILE* file = fopen(filename, "r");
    if (!file) {
        printf("Error: Could not open file.\n");
        return 0;
    }
```

```

    }

    int count = 0;
    while (fscanf(file, "%s %d %s", students[count].name,
&students[count].rollNo, students[count].department) == 3) {
        count++;
        if (count >= MAX_STUDENTS) break;
    }
    fclose(file);
    return count;
}

void bubbleSort(Student students[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (strcmp(students[j].name, students[j + 1].name) > 0 ||
                (strcmp(students[j].name, students[j + 1].name) == 0 &&
students[j].rollNo > students[j + 1].rollNo)) {
                Student temp = students[j];
                students[j] = students[j + 1];
                students[j + 1] = temp;
            }
        }
    }
}

void binarySearch(Student students[], int n, const char* targetName) {
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (strcmp(students[i].name, targetName) == 0) {
            printf("Found: %s %d %s\n", students[i].name, students[i].rollNo,
students[i].department);
            found = 1;
        }
    }
    if (!found) {
        printf("No records found for student name: %s\n", targetName);
    }
}

int main() {
    Student students[MAX_STUDENTS];
    int studentCount = readFromFile(students, "students.txt");

    if (studentCount == 0) {
        return 1;
    }
}

```

```

    int choice;
    do {
        printf("Choose an option:\n1. Bubble Sort\n2. Binary Search\n3.
Quit\n");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                bubbleSort(students, studentCount);
                printf("Records after sorting:\n");
                for (int i = 0; i < studentCount; i++) {
                    printf("%s %d %s\n", students[i].name, students[i].rollNo,
students[i].department);
                }
                break;
            case 2: {
                char name[NAME_SIZE];
                printf("Enter the name to search: ");
                scanf("%s", name);
                binarySearch(students, studentCount, name);
                break;
            }
            case 3:
                printf("Exiting program...\n");
                break;
            default:
                printf("Invalid option. Try again.\n");
        }
    } while (choice != 3);

    return 0;
}

```

```

≡ students.txt
1   John 101 CSE
2   Mary 102 ECE
3   Peter 103 ME
4   Alice 104 IT
5   John 105 EEE
6   Kate 106 CSE

```

Output:

```
Choose an option:
1. Bubble Sort
2. Binary Search
3. Quit
1
Records after sorting:
Alice 104 IT
John 101 CSE
John 105 EEE
Kate 106 CSE
Mary 102 ECE
Peter 103 ME
Choose an option:
1. Bubble Sort
2. Binary Search
3. Quit
2
Enter the name to search: Alice
Found: Alice 104 IT
Choose an option:
1. Bubble Sort
2. Binary Search
3. Quit
3
Exiting program...
```

2. Let $A[n]$ be an array of n distinct integers. If $i < j$ and $A[i] > A[j]$, then the pair (i, j) is called an inversion of A . Write a C/C++ program that determines the number of Inversions in any permutation on n elements in $O(n \lg n)$ worst-case time. (Hint: Modify merge sort)

Code:

```
#include <iostream>
using namespace std;

int count=0;
void merge(int arr[], int low, int mid, int high) {
    int temp[high-low+1];
    int i=low;
    int j=mid+1;
    int k=0;

    while (i<=mid && j<=high) {
        if (arr[i]<=arr[j]) {
            temp[k++]=arr[i++];
        }

        else {
            temp[k++]=arr[j++];
            count+=mid-i+1;
        }
    }

    while (j<=high) {
        temp[k++]=arr[j++];
    }

    while (i<=mid) {
        temp[k++]=arr[i++];
    }

    for (k=0, i=low; i<=high; k++, i++) {
        arr[i]=temp[k];
    }
}

void mergeSort(int arr[], int low, int high) {
    if (low<high) {
        int mid=low+(high-low)/2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid+1, high);
    }
}
```

```

        merge(arr, low, mid, high);
    }
}

void printArray(int arr[], int n) {
    for (int i=0; i<n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter the elements: ";
    for (int i=0; i<n; i++) {
        cin >> arr[i];
    }

    cout << "Array: ";
    printArray(arr, n);

    mergeSort(arr, 0, n-1);
    cout << "Number of Inversions: " << count << endl;
}

```

Output:

```

Enter the number of elements: 4
Enter the elements: 4 1 3 2
Array: 4 1 3 2
Number of Inversions: 4

```